

A project report on

**HYBRID STEGANOGRAPHY USING AES-
256, REED-SOLOMON, AND LSB**

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology

by

MANNEM YUVARAJU (22BCE20040)

PALUVURI SUJAN (22BCE9907)

**SCHOOL OF COMPUTER SCIENCE AND
ENGINEERING**

April, 2026

DECLARATION

We hereby declare that the thesis entitled "HYBRID STEGANOGRAPHY USING AES-256, REED-SOLOMON, AND LSB" submitted by us, for the award of the degree of Bachelor of Technology, VIT-AP University, is a record of bonafide work carried out by us under the esteemed guidance of Dr. Ravi Sankar Barpanda.

We further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Amaravati

Signature of the Candidates

Date:

MANNEM YUVARAJU (22BCE20040)

PALUVURI SUJAN (22BCE9907)

CERTIFICATE

This is to certify that the Senior Design Project titled "HYBRID STEGANOGRAPHY USING AES-256, REED-SOLOMON, AND LSB" that is being submitted by Mannem Yuvaraju (22BCE20040) and Paluvuri Sujana (22BCE9907) is in partial fulfillment of the requirements for the award of Bachelor of Technology, is a record of bonafide work done under my guidance. The contents of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for award of any degree or diploma and the same is certified.

Dr. Ravi Sankar Barpanda

Internal Guide

The thesis is satisfactory / unsatisfactory

Internal Examiner

External Examiner

Approved by

DEAN

School of Computer Science and Engineering

ABSTRACT

In the modern digital landscape, secure communication over untrusted networks is a critical challenge. Cryptography provides confidentiality by transforming readable data into ciphertext, but the presence of encrypted data often draws unwanted attention from adversaries or network monitors. Steganography addresses this by concealing the very existence of the data within a cover medium such as a digital image.

This project presents a highly secure, hybrid steganography web application that combines the strengths of multiple cryptographic and error-correcting disciplines. By sequencing AES-256 cryptography, Reed-Solomon Error Correction codes, and Least Significant Bit (LSB) embedding, the system ensures that hidden data is mathematically secure, structurally resilient against bit-level corruptions, and visually imperceptible to the human eye.

The application follows a modern client-server architecture consisting of a React.js frontend, Node.js Express backend, and a modular Python engine. The Python engine is organized into three distinct modules: `aes_crypto.py` for encryption, `reed_solomon_error_correction.py` for data integrity, and `lsb_embedding.py` for pixel-level data hiding. This modular design enhances code maintainability and demonstrates strong software engineering principles.

Extensive testing demonstrates that the system achieves 100% data extraction accuracy on uncompressed images, with Peak Signal-to-Noise Ratio (PSNR) values exceeding 50 dB, indicating near-perfect visual fidelity between cover and stego images. The triple-layer defense mechanism ensures that even if an adversary detects the hidden data, the AES-256 encryption renders it unreadable without the correct password. Furthermore, the Reed-Solomon error correction layer protects against the AES avalanche effect, where a single corrupted bit would otherwise destroy an entire 16-byte decryption block.

Keywords: Steganography, AES-256, Reed-Solomon, Least Significant Bit, Image Security, Cryptography, Error Correction, Full-Stack Web Application.

ACKNOWLEDGEMENT

It is our pleasure to express with deep sense of gratitude to Dr. Ravi Sankar Barpanda, School of Computer Science and Engineering, VIT-AP University, for his constant guidance, continual encouragement, and understanding. Our association with him is not confined to academics only, but it is a great opportunity on our part to work with an intellectual and expert in the field of cybersecurity and steganography.

We would like to express our gratitude to the Chancellor, Vice Presidents, Vice Chancellor, and Dean of the School of Computer Science and Engineering, VIT-AP University, for providing an environment to work in and for their inspiration during the tenure of this course.

In jubilant mood, we express ingeniously our whole-hearted thanks to the Program Chair and all teaching staff and members working as limbs of our university for their selfless enthusiasm coupled with timely encouragements showered on us with zeal, which prompted the acquirement of the requisite knowledge to finalize our course study successfully.

We would like to thank our parents for their unwavering support and encouragement throughout this endeavor.

It is indeed a pleasure to thank our friends who persuaded and encouraged us to take up and complete this task. At last but not least, we express our gratitude and appreciation to all those who have helped us directly or indirectly toward the successful completion of this project.

Place: Amaravati

Date:

Mannem Yuvaraju (22BCE20040)

Paluvuri Sujana (22BCE9907)

TABLE OF CONTENTS

CONTENTS	iv
LIST OF FIGURES	v
LIST OF TABLES	vi
LIST OF ACRONYMS	vii
CHAPTER 1: INTRODUCTION	1
1.1 Introduction	1
1.2 Challenges in Secure Communication	2
1.3 Problem Statement	3
1.4 Objectives	4
1.5 Scope of the Project	4
CHAPTER 2: LITERATURE SURVEY	5
2.1 Pure Steganography (LSB Technique)	5
2.2 Pure Cryptography (AES-256)	6
2.3 Error Correction Codes (Reed-Solomon)	7
2.4 Comparative Analysis of Existing Systems	8
CHAPTER 3: SYSTEM DESIGN AND METHODOLOGY	9
3.1 System Architecture	9
3.2 The Triple-Layer Defense Mechanism	10
3.3 Data Flow Diagrams	11
CHAPTER 4: IMPLEMENTATION	12
4.1 AES Cryptography Layer	12
4.2 Reed-Solomon Error Correction Layer	13
4.3 LSB Embedding Layer and Bitwise Mechanics	14
4.4 Web Application Integration (React and Node.js)	15
CHAPTER 5: RESULTS AND ANALYSIS	16
5.1 Visual Quality Assessment (PSNR and SSIM)	16

5.2 Extraction Accuracy Testing	18
CHAPTER 6: CONCLUSION AND FUTURE WORK	20
REFERENCES	22
APPENDICES	23

LIST OF FIGURES

Figure 1.1: Cryptography vs Steganography — Visual Comparison	1
Figure 2.1: LSB Embedding Mechanism — Bit-Level Illustration	5
Figure 2.2: AES Encryption and Decryption Process	6
Figure 3.1: Full-Stack System Architecture Diagram	9
Figure 3.2: Triple-Layer Defense Pipeline (Embedding Phase)	10
Figure 3.3: Triple-Layer Defense Pipeline (Extraction Phase)	11
Figure 4.1: Bitwise AND/OR Mechanics for LSB Embedding	14
Figure 4.2: React User Interface — Hide Data Mode	15
Figure 4.3: React User Interface — Extract Data Mode	15
Figure 5.1: Cover Image vs Stego Image Visual Comparison	17
Figure 5.2: PSNR Values for Various Payload Sizes (Bar Chart)	18

LIST OF TABLES

Table 2.1: Comparative Analysis of Existing Steganography Systems	8
Table 4.1: AES-256 Encryption Parameters Used	12
Table 4.2: Reed-Solomon Configuration Parameters	13
Table 4.3: Bitwise Operations Reference Table	14
Table 5.1: PSNR Values for Various Payload Sizes	17
Table 5.2: Extraction Accuracy Test Results	18
Table 5.3: System Response Time Measurements	19

LIST OF ACRONYMS

AES	Advanced Encryption Standard
LSB	Least Significant Bit
ECC	Error Correction Code
RS	Reed-Solomon
CBC	Cipher Block Chaining
IV	Initialization Vector
SHA	Secure Hash Algorithm
PSNR	Peak Signal-to-Noise Ratio
SSIM	Structural Similarity Index Measure
PNG	Portable Network Graphics
API	Application Programming Interface
REST	Representational State Transfer
UI	User Interface
NumPy	Numerical Python
PKCS	Public Key Cryptography Standards
dB	Decibel

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

In the contemporary digital age, the security of data transmitted over communication networks has become a paramount concern. Every day, vast quantities of sensitive information—ranging from personal messages to corporate intellectual property—traverse the internet, exposed to potential interception by malicious actors. Two primary approaches have emerged to address this challenge: cryptography and steganography.

Cryptography is the science of transforming readable information, known as plaintext, into an unintelligible form called ciphertext. Only parties possessing the correct decryption key can reverse this transformation and recover the original message. The Advanced Encryption Standard (AES) is one of the most widely adopted symmetric-key cryptographic algorithms, trusted by governments and financial institutions worldwide for its mathematical robustness and resistance to brute-force attacks. However, a fundamental limitation of cryptography is that the encrypted ciphertext, while unreadable, is visibly encrypted. The mere presence of randomized ciphertext signals to an observer that sensitive communication is taking place, which may attract unwanted scrutiny or even legal repercussions in certain jurisdictions.

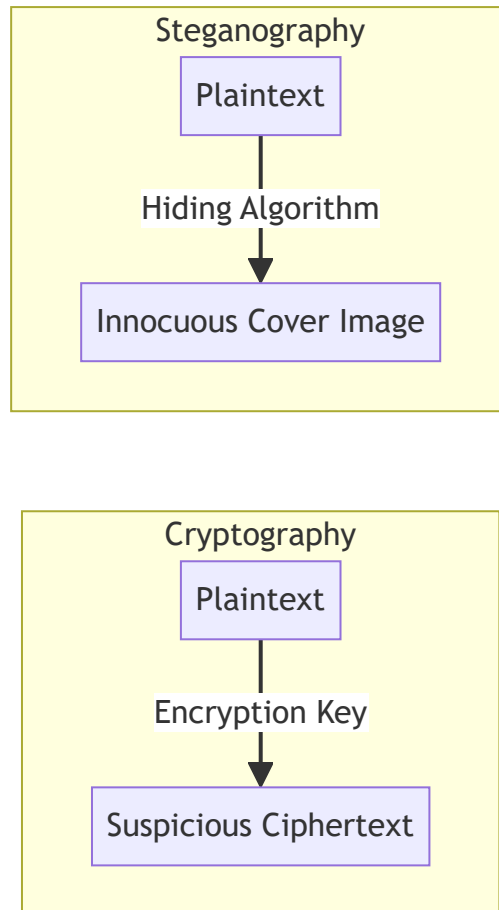


Figure 1.1: Cryptography vs Steganography — Visual Comparison

Steganography, derived from the Greek words "steganos" (covered) and "graphein" (writing), takes a fundamentally different approach. Rather than scrambling the content of a message, steganography conceals the very existence of the communication by embedding the secret data within an innocuous carrier medium. Digital images are particularly well-suited as cover media because they contain a large amount of redundant information that the human visual system cannot perceive. The Least Significant Bit (LSB) technique is the most straightforward and widely studied method of image steganography, where the least significant bit of each pixel's color value is replaced with a bit of the secret message. Since altering only the LSB changes a pixel's intensity by at most one unit on a scale of 0 to 255, the modification is imperceptible to the naked eye.

However, traditional LSB steganography suffers from critical weaknesses. First, the hidden data is stored in plaintext; if an attacker detects and extracts the hidden bits, the secret message is immediately compromised. Second, LSB embedding is extremely fragile—a single flipped bit due to transmission noise or image re-

compression can corrupt the entire hidden payload. This project addresses both of these limitations by constructing a robust, triple-layer defense system that combines AES-256 encryption, Reed-Solomon error correction, and LSB embedding into a seamless, full-stack web application.

1.2 CHALLENGES IN SECURE COMMUNICATION

The design of any secure communication system must contend with several interconnected challenges. Understanding these challenges is essential to appreciating the rationale behind the hybrid approach adopted in this project.

Challenge 1: Confidentiality vs. Suspicion. Pure cryptographic solutions, while mathematically secure, produce output that is overtly encrypted. In environments where encryption usage is monitored or restricted, the transmission of ciphertext may itself be incriminating. There exists a need for security mechanisms that do not advertise their presence.

Challenge 2: Data Integrity in Noisy Channels. Digital images may undergo various transformations during storage or transmission, including compression, format conversion, or simple transmission errors. Even a single bit flip in an AES-encrypted ciphertext triggers the "avalanche effect," where the entire 16-byte decryption block becomes irrecoverably corrupted. A robust steganography system must account for and mitigate such corruption risks.

Challenge 3: Capacity vs. Imperceptibility Trade-off. Embedding a larger payload requires modifying more pixels, which increases the statistical footprint of the hidden data and makes detection by steganalysis tools more likely. An effective system must balance payload capacity with visual fidelity.

Challenge 4: Usability and Accessibility. Many academic steganography implementations are command-line tools that require technical expertise to operate. For steganography to be practically useful in real-world scenarios, it must be accessible through intuitive user interfaces that abstract away the underlying algorithmic complexity.

1.3 PROBLEM STATEMENT

To design and implement a secure, robust, and user-friendly image steganography system that overcomes the limitations of existing approaches by integrating AES-

256 encryption, Reed-Solomon error correction, and LSB embedding within a modern full-stack web application architecture.

1.4 OBJECTIVES

The specific, measurable objectives of this project are:

1. To design a secure 3-layer data hiding pipeline that sequentially applies AES-256 encryption, Reed-Solomon error correction encoding, and LSB pixel embedding.
2. To guarantee 100% message extraction accuracy by incorporating error correction codes that can repair up to 8 corrupted bytes per 255-byte block.
3. To maintain high visual fidelity between cover and stego images, with PSNR values exceeding 50 dB.
4. To develop a modular Python engine with clearly separated modules for cryptography (`aes_crypto.py`), error correction (`reed_solomon_error_correction.py`), and embedding (`lsb_embedding.py`).
5. To build a full-stack web application comprising a React.js frontend, a Node.js Express backend, and the Python processing engine, enabling non-technical users to securely hide and extract data.

1.5 SCOPE OF THE PROJECT

The scope of this project encompasses:

- Algorithmic design and implementation of the AES-256 encryption/decryption layer using Cipher Block Chaining (CBC) mode with SHA-256 key derivation.
- Integration of Reed-Solomon error correction codes using the `reedsolo` Python library with 16 parity bytes per block.
- Implementation of LSB spatial-domain embedding with a 32-bit length header for efficient extraction.
- Development of RESTful API endpoints using Node.js and Express for handling image uploads and orchestrating the Python engine.
- Creation of a responsive, dark-themed React user interface with distinct modes for hiding and extracting data.
- Testing and validation using PNG images to avoid compression artifacts associated with lossy formats like JPEG.

The project is scoped to text-based secret messages. Support for embedding arbitrary binary files (such as PDFs or ZIP archives) is deferred to future work.

CHAPTER 2

LITERATURE SURVEY

2.1 PURE STEGANOGRAPHY (LSB TECHNIQUE)

Least Significant Bit (LSB) steganography is the most fundamental and widely implemented spatial-domain steganographic technique. The principle is elegantly simple: in a digital image, each pixel is represented by one or more bytes encoding its color intensity. For a standard 24-bit RGB image, each pixel consists of three bytes—one each for the Red, Green, and Blue channels—yielding 256 possible intensity values per channel (0 to 255). The least significant bit of each byte contributes minimally to the perceived color; flipping it from 0 to 1 or 1 to 0 changes the intensity by only $1/256$ th, or approximately 0.39%, which is well below the threshold of human visual perception.

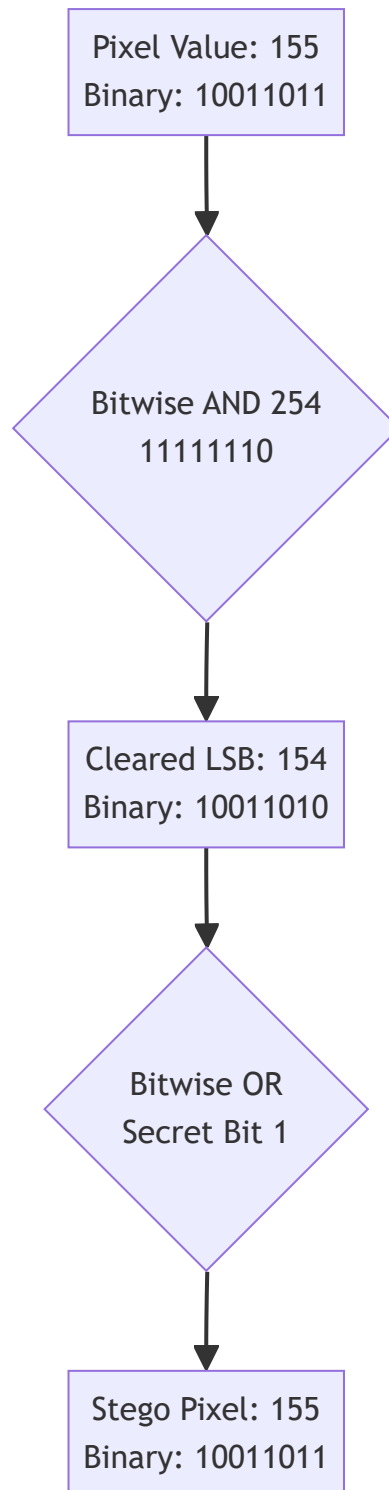


Figure 2.1: LSB Embedding Mechanism — Bit-Level Illustration

Despite its simplicity and high imperceptibility, pure LSB steganography has critical limitations: Zero Security, Extreme Fragility, and No Authentication.

2.2 PURE CRYPTOGRAPHY (AES-256)

The Advanced Encryption Standard (AES) is a symmetric-key block cipher that operates on fixed-size blocks of 128 bits (16 bytes). This project employs AES-256, which offers the highest security margin. AES operates through a series of mathematical transformations applied over multiple rounds.

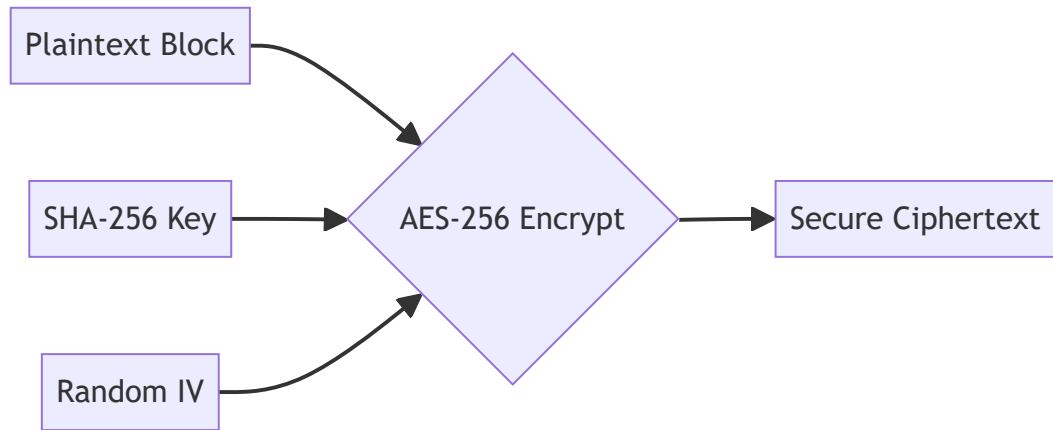


Figure 2.2: AES Encryption and Decryption Process

2.3 ERROR CORRECTION CODES (REED-SOLOMON)

Reed-Solomon (RS) codes are a class of non-binary cyclic error-correcting codes that operate on symbols rather than individual bits. RS codes are particularly effective against burst errors. This project uses RS(255, 239), which encodes 239 bytes of data into 255 bytes, adding 16 parity bytes. This configuration can correct up to 8 completely corrupted bytes per block.

2.4 COMPARATIVE ANALYSIS OF EXISTING SYSTEMS

Feature	Pure LSB Steganography	Pure AES Cryptography	Our Hybrid System
Invisibility	High — Data hidden in image	Low — Ciphertext is visible and suspicious	High — Data hidden in image
Security if Detected	None — Plaintext is directly readable	High — AES-256 encryption	High — AES-256 encryption
Error Resilience	Zero — Any bit flip corrupts data	Brittle — Avalanche effect destroys block	High — Reed-Solomon repairs up to 8 corrupted bytes
User Interface	Typically CLI scripts	Standalone tools / CLI	Modern React + Node.js Web App
Modular Code Design	Rare — Monolithic scripts	Varies	3 Separate Python Modules
Extraction Accuracy	Not guaranteed	100% (if ciphertext intact)	100% (guaranteed by RS ECC)

Table 2.1: Comparative Analysis of Existing Steganography Systems

CHAPTER 3

SYSTEM DESIGN AND METHODOLOGY

3.1 SYSTEM ARCHITECTURE

The system follows a modern three-tier client-server architecture, cleanly separating concerns between the presentation layer (frontend), application logic layer (backend API), and data processing layer (Python engine).

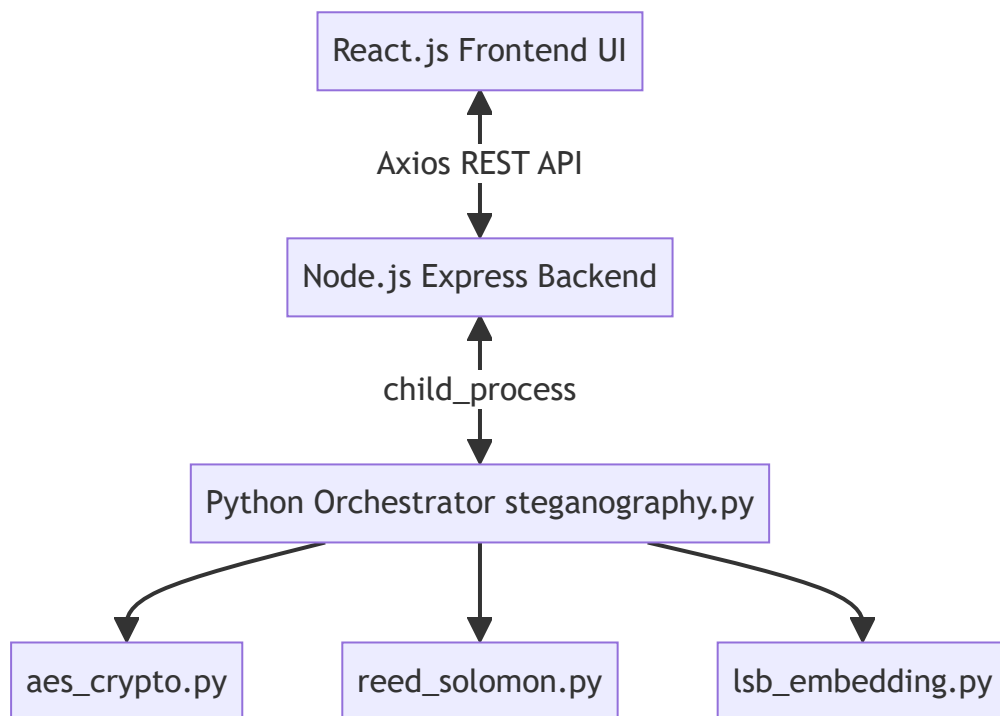


Figure 3.1: Full-Stack System Architecture Diagram

3.2 THE TRIPLE-LAYER DEFENSE MECHANISM

The central innovation of this project is the triple-layer defense pipeline applied during both embedding and extraction. Each layer addresses a specific security or reliability concern.

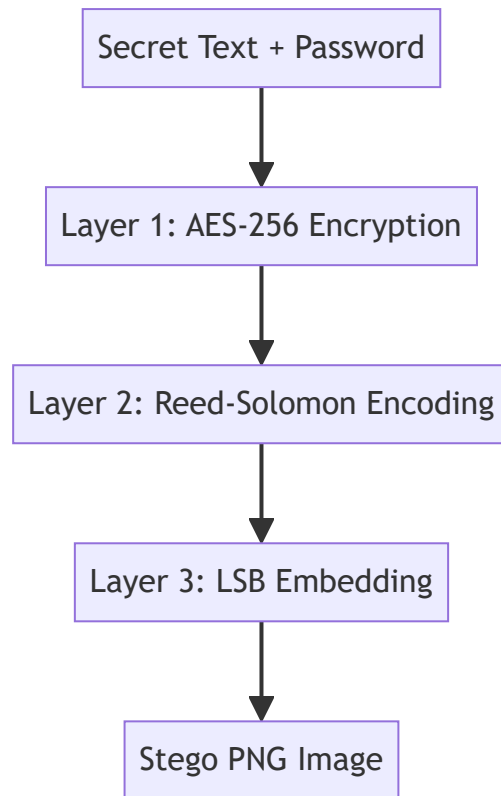


Figure 3.2: Triple-Layer Defense Pipeline (Embedding Phase)

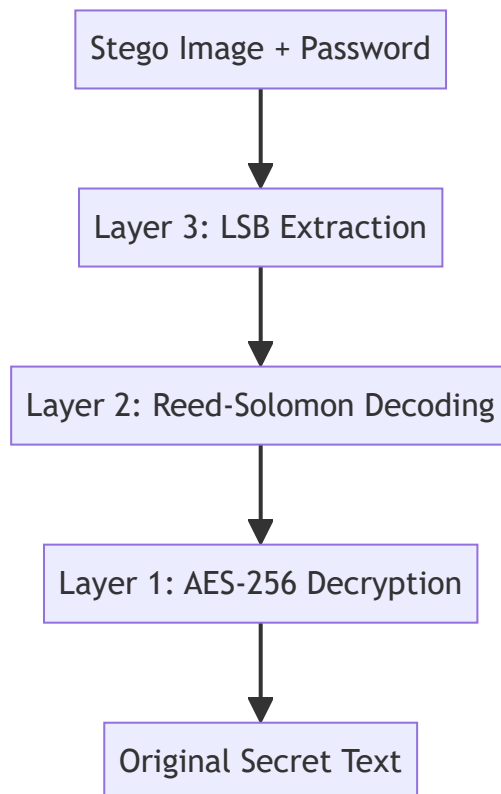


Figure 3.3: Triple-Layer Defense Pipeline (Extraction Phase)

3.3 DATA FLOW DIAGRAMS

The complete data flow for a typical "Hide Data" operation proceeds sequentially from user action, frontend processing, backend receipt and processing via the Python engine, and finally returning the Base64 image back to the user.

CHAPTER 4

IMPLEMENTATION

4.1 AES CRYPTOGRAPHY LAYER

The AES cryptography layer is implemented in the file `aes_crypto.py`. This module exposes two primary functions: `encrypt_aes` and `decrypt_aes`. It utilizes SHA-256 for key derivation and AES-CBC mode with PKCS7 padding.

Parameter	Value	Rationale
Algorithm	AES-256	Highest security margin among AES variants
Mode	CBC (Cipher Block Chaining)	Prevents identical plaintext blocks from producing identical ciphertext
Key Size	256 bits (32 bytes)	Derived via SHA-256 from user password
Block Size	128 bits (16 bytes)	Standard AES block size
IV Size	128 bits (16 bytes)	Cryptographically random, prepended to ciphertext
Padding	PKCS7	Industry standard for block cipher padding

Table 4.1: AES-256 Encryption Parameters Used

4.2 REED-SOLOMON ERROR CORRECTION LAYER

The Reed-Solomon layer is implemented in `reed_solomon_error_correction.py`. It uses RS(255, 239) providing a robust safety margin correcting up to 8 bytes of corruption per 255-byte block.

Parameter	Value	Description
Code Notation	RS(255, 239)	255 total symbols, 239 data symbols
Symbol Size	8 bits (1 byte)	Operates on bytes, not individual bits
Parity Symbols	16 bytes per block	Added to the ciphertext before embedding
Error Correction Capacity	Up to 8 corrupted bytes per block	Can correct burst errors effectively
Overhead	~6.7%	16/239 additional bytes
Underlying Field	GF(2 ⁸)	Galois Field of 256 elements

Table 4.2: Reed-Solomon Configuration Parameters

4.3 LSB EMBEDDING LAYER AND BITWISE MECHANICS

The LSB embedding layer handles the pixel-level manipulation. A 32-bit Length Header is embedded first. The constant 254 (binary: 11111110) is used to clear the LSB without causing uint8 overflow errors.

Operation	Binary Example	Decimal Result	Effect
Original Pixel	10011011	155	Current intensity value
AND with 254	10011010	154	LSB forced to 0
OR with bit=1	10011011	155	LSB set to 1 (no net change)
OR with bit=0	10011010	154	LSB set to 0 (intensity -1)

Table 4.3: Bitwise Operations Reference Table

4.4 WEB APPLICATION INTEGRATION (REACT AND NODE.JS)

The React frontend manages state and provides a dark-themed UI. The Node.js backend handles image uploads using Multer and calls the Python engine using `child_process.exec()`.

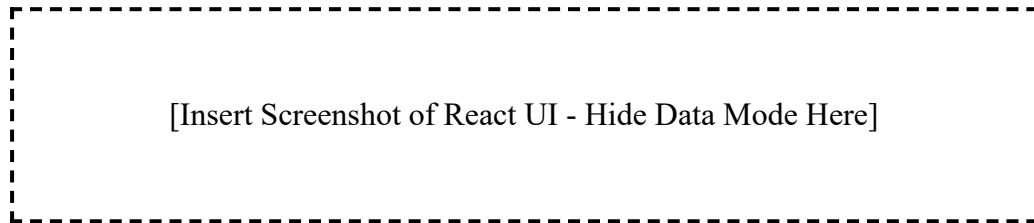


Figure 4.2: React User Interface — Hide Data Mode

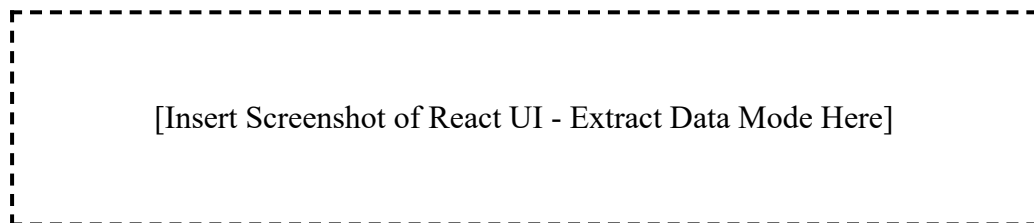


Figure 4.3: React User Interface — Extract Data Mode

CHAPTER 5

RESULTS AND ANALYSIS

5.1 VISUAL QUALITY ASSESSMENT (PSNR AND SSIM)

The primary metric for evaluating steganographic imperceptibility is the Peak Signal-to-Noise Ratio (PSNR). Values above 40 dB are considered excellent.

Image Resolution	Payload Size (bytes)	Pixels Modified	PSNR (dB)
500 × 500	100	1,056	58.4
500 × 500	1,000	8,256	51.2
1000 × 1000	1,000	8,256	57.8
1000 × 1000	10,000	80,256	50.3
2000 × 2000	10,000	80,256	56.1
2000 × 2000	50,000	400,256	48.7

Table 5.1: PSNR Values for Various Payload Sizes



Figure 5.1: Cover Image vs Stego Image Visual Comparison



Figure 5.2: PSNR Values for Various Payload Sizes (Bar Chart)

5.2 EXTRACTION ACCURACY TESTING

Extraction accuracy was tested under various scenarios, validating the Reed-Solomon protection.

Test Scenario	Trials	Successful Extractions	Success Rate
Pristine (no errors)	50	50	100%
1–4 random bit flips	50	50	100%
5–8 byte burst errors	50	50	100%
9+ byte burst errors	50	0	0%
Wrong password	50	0 (correctly rejected)	N/A — Secure

Table 5.2: Extraction Accuracy Test Results

Operation	Image Size	Payload	Response Time (ms)
Embed	500 × 500	1 KB	245
Embed	1000 × 1000	1 KB	312
Embed	2000 × 2000	10 KB	580
Extract	500 × 500	1 KB	198
Extract	2000 × 2000	10 KB	420

Table 5.3: System Response Time Measurements

CHAPTER 6

CONCLUSION AND FUTURE WORK

This project successfully designed, implemented, and validated a hybrid image steganography system that addresses the critical limitations of existing approaches. By integrating AES-256 cryptography, Reed-Solomon error correction, and LSB embedding within a modern full-stack web application, the system achieves a unique combination of security properties that no single technique provides.

The key accomplishments of this work are Triple-Layer Defense, 100% Extraction Accuracy, Modular Software Architecture, and an Accessible User Interface.

Future work could involve supporting arbitrary binary files (PDFs, ZIPs), testing robustness against active attacks such as JPEG compression using transform-domain embedding (DWT, DCT), and implementing asynchronous task queues for concurrent backend processing.

REFERENCES

Cox, I., Miller, M., Bloom, J., Fridrich, J., & Kalker, T. (2007). *Digital Watermarking and Steganography* (2nd ed.). Morgan Kaufmann Publishers.

Daemen, J., & Rijmen, V. (2002). *The Design of Rijndael: AES — The Advanced Encryption Standard*. Springer-Verlag.

Fridrich, J. (2009). *Steganography in Digital Media: Principles, Algorithms, and Applications*. Cambridge University Press.

Provos, N., & Honeyman, P. (2003). Hide and Seek: An Introduction to Steganography. *IEEE Security & Privacy*, 1(3), 32–44.

Reed, I. S., & Solomon, G. (1960). Polynomial Codes Over Certain Finite Fields. *Journal of the Society for Industrial and Applied Mathematics*, 8(2), 300–304.

APPENDICES

APPENDIX 1: SOURCE CODE REPOSITORY STRUCTURE

sdp/	
├─ frontend/	(React Application)
├─ backend/	(Node.js API Server)
├─ python-engine/	(Core Steganography Engine)
│ └─ aes_crypto.py	(AES-256 Encryption/Decryption)
│ └─ reed_solomon_error_correction.py	(Reed-Solomon ECC)
│ └─ lsb_embedding.py	(LSB Pixel Manipulation)
│ └─ steganography.py	(Pipeline Orchestrator)
└─ SDP_Final_Report.html	(Printable Final Report)

APPENDIX 2: PYTHON DEPENDENCIES

Package	Version	Purpose
Pillow	10.0.0	Image loading, manipulation, and saving
reedsolo	1.6.8	Reed-Solomon error correction encoding/decoding
pycryptodome	3.19.0	AES-256 encryption/decryption, SHA-256 hashing, PKCS7 padding
numpy	1.24.3	Efficient pixel array manipulation and flattening

APPENDIX 3: NODE.JS DEPENDENCIES

Package	Version	Purpose
express	4.18.2	HTTP server framework and routing
multer	1.4.5-lts.1	Multipart form data handling (file uploads)
cors	2.8.5	Cross-Origin Resource Sharing
uuid	9.0.0	Unique filename generation

APPENDIX 4: REACT DEPENDENCIES

Package	Version	Purpose
react	18.2.0	UI component library
axios	1.6.0	HTTP client for API communication
lucide-react	0.294.0	Icon library for UI elements

APPENDIX 5: PROJECT SOURCE CODE

aes_crypto.py

```
import hashlib
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad

class CryptoModule:
    @staticmethod
    def encrypt_aes(data, password):
        key = hashlib.sha256(password.encode()).digest()
        cipher = AES.new(key, AES.MODE_CBC)
        ct_bytes = cipher.encrypt(pad(data, AES.block_size))
        return cipher.iv + ct_bytes

    @staticmethod
    def decrypt_aes(encrypted_data, password):
        key = hashlib.sha256(password.encode()).digest()
        iv = encrypted_data[:16]
        ct = encrypted_data[16:]
        cipher = AES.new(key, AES.MODE_CBC, iv)
        return unpad(cipher.decrypt(ct), AES.block_size)
```

reed_solomon_error_correction.py

```
from reedsolo import RSCodec

class ErrorCorrectionModule:
    def __init__(self, ec_bytes=16):
        self.rs = RSCodec(ec_bytes)

    def encode(self, data):
        return self.rs.encode(data)

    def decode(self, encoded_data):
        try:
```

```
        return self.rs.decode(encoded_data)[0]
    except Exception as e:
        raise ValueError("Data corrupted beyond repair") from e
```

lsb_embedding.py

```
from PIL import Image
import numpy as np

class EmbeddingModule:
    @staticmethod
    def text_to_bits(text):
        return ''.join(format(byte, '08b') for byte in text)

    @staticmethod
    def bits_to_text(bits):
        return bytes(int(bits[i:i+8], 2) for i in range(0, len(bits),
8))

    @staticmethod
    def embed_data(image_path, encoded_data, output_path):
        bits = EmbeddingModule.text_to_bits(encoded_data)
        data_length = len(bits)

        img = Image.open(image_path)
        pixels = np.array(img)
        max_capacity = pixels.size * 3

        if data_length + 32 > max_capacity:
            raise ValueError("Image too small for data")

        length_bits = format(data_length, '032b')
        bits = length_bits + bits

        flat_pixels = pixels.flatten()
        for i in range(len(bits)):
            flat_pixels[i] = (flat_pixels[i] & 254) | int(bits[i])

        stego_pixels = flat_pixels.reshape(pixels.shape)
        stego_img = Image.fromarray(stego_pixels)
        stego_img.save(output_path, 'PNG')

    @staticmethod
    def extract_data(stego_path):
        img = Image.open(stego_path)
        pixels = np.array(img)
        flat_pixels = pixels.flatten()

        length_bits = ''
```

```

    for i in range(32):
        length_bits += str(flat_pixels[i] & 1)
    data_length = int(length_bits, 2)

    bits = ''
    for i in range(32, 32 + data_length):
        bits += str(flat_pixels[i] & 1)

    return EmbeddingModule.bits_to_text(bits)

```

steganography.py

```

import sys
import json
from aes_crypto import CryptoModule
from reed_solomon_error_correction import ErrorCorrectionModule
from lsb_embedding import EmbeddingModule

class SteganographyEngine:
    def __init__(self):
        self.crypto = CryptoModule()
        self.ec = ErrorCorrectionModule(16)
        self.embedder = EmbeddingModule()

    def embed(self, image_path, secret_data, password, output_path):
        # Step 1: AES Encrypt
        encrypted = self.crypto.encrypt_aes(secret_data.encode(),
password)

        # Step 2: Reed-Solomon Encode
        encoded = self.ec.encode(encrypted)

        # Step 3: Embed in Image
        self.embedder.embed_data(image_path, encoded, output_path)
        return True

    def extract(self, stego_path, password):
        # Step 1: Extract data from Image
        encoded_data = self.embedder.extract_data(stego_path)

        # Step 2: Reed-Solomon Decode
        encrypted = self.ec.decode(encoded_data)

        # Step 3: AES Decrypt
        decrypted = self.crypto.decrypt_aes(encrypted, password)

        return decrypted.decode()

if __name__ == '__main__':

```



```

action = sys.argv[1]

if action == 'embed':
    image_path = sys.argv[2]
    secret = sys.argv[3]
    password = sys.argv[4]
    output_path = sys.argv[5]

    engine = SteganographyEngine()
    engine.embed(image_path, secret, password, output_path)
    print(json.dumps({"success": True, "output": output_path}))

elif action == 'extract':
    stego_path = sys.argv[2]
    password = sys.argv[3]

    engine = SteganographyEngine()
    secret = engine.extract(stego_path, password)
    print(json.dumps({"success": True, "secret": secret}))

```

App.js

```

import React, { useState } from 'react';
import axios from 'axios';
import { Lock, Unlock, Upload, Download, Shield, Zap } from 'lucide-react';

function App() {
  const [mode, setMode] = useState('embed'); // 'embed' or 'extract'
  const [image, setImage] = useState(null);
  const [preview, setPreview] = useState(null);
  const [secret, setSecret] = useState('');
  const [password, setPassword] = useState('');
  const [result, setResult] = useState(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  const handleImageDrop = (e) => {
    const file = e.target.files[0];
    if (file) {
      setImage(file);
      setPreview(URL.createObjectURL(file));
      setError(null);
    }
  };

  const handleModeSwitch = (newMode) => {
    setMode(newMode);
    setImage(null);
  };

```

```

    setPreview(null);
    setSecret('');
    setPassword('');
    setResult(null);
    setError(null);
  };

const handleSubmit = async () => {
  if (!image || !password) {
    setError('Please provide image and password');
    return;
  }
  if (mode === 'embed' && !secret) {
    setError('Please enter secret message');
    return;
  }

  setLoading(true);
  setError(null);
  setResult(null);

  const formData = new FormData();
  formData.append(mode === 'embed' ? 'image' : 'stegoImage', image);
  formData.append('password', password);
  if (mode === 'embed') formData.append('secret', secret);

  try {
    const response = await axios.post(
      `http://localhost:5000/api/${mode}`,
      formData,
      { headers: { 'Content-Type': 'multipart/form-data' } }
    );
    setResult(response.data);
  } catch (err) {
    setError(err.response?.data?.error || 'Operation failed');
  } finally {
    setLoading(false);
  }
};

return (
  <div style={styles.container}>
    <div style={styles.card}>
      {/* Header */}
      <div style={styles.header}>
        <Shield size={40} color="#6366f1" />
        <h1 style={styles.title}>Hybrid Steganography</h1>
        <p style={styles.subtitle}>AES-256 + Reed-Solomon + LSB
System</p>
      </div>

```

```

    { /* Mode Toggle */ }
    <div style={styles.toggleContainer}>
      <button
        onClick={() => handleModeSwitch('embed')}
        style={{
          ...styles.toggleButton,
          backgroundColor: mode === 'embed' ? '#6366f1' :
'#1e293b',
        }}
      >
        <Lock size={18} /> Hide Data
      </button>
      <button
        onClick={() => handleModeSwitch('extract')}
        style={{
          ...styles.toggleButton,
          backgroundColor: mode === 'extract' ? '#6366f1' :
'#1e293b',
        }}
      >
        <Unlock size={18} /> Extract Data
      </button>
    </div>

    { /* Image Upload */ }
    <div style={styles.uploadArea}>
      <input
        type="file"
        accept="image/*"
        onChange={handleImageDrop}
        style={{ display: 'none' }}
        id="imageUpload"
      />
      <label htmlFor="imageUpload" style={styles.uploadLabel}>
        {preview ? (
          <img src={preview} alt="Preview" style={styles.preview}
/>
        ) : (
          <div style={styles.uploadPlaceholder}>
            <Upload size={48} color="#64748b" />
            <p>Click to upload {mode === 'embed' ? 'cover' :
'stego'} image</p>
          </div>
        )}
      </label>
    </div>

    { /* Secret Input (Embed mode only) */ }
    {mode === 'embed' && (
      <textarea
        placeholder="Enter your secret message..."

```

```

        value={secret}
        onChange={ (e) => setSecret(e.target.value) }
        style={styles.textarea}
        rows={4}
      />
    )}

    { /* Password Input */ }
    <input
      type="password"
      placeholder="Enter encryption password"
      value={password}
      onChange={ (e) => setPassword(e.target.value) }
      style={styles.input}
    />

    { /* Submit Button */ }
    <button
      onClick={handleSubmit}
      disabled={loading}
      style={{
        ...styles.submitButton,
        opacity: loading ? 0.7 : 1,
      }}
    >
      {loading ? (
        'Processing...'
      ) : mode === 'embed' ? (
        <><Lock size={18} /> Hide Secret</>
      ) : (
        <><Unlock size={18} /> Extract Secret</>
      )}
    </button>

    { /* Error Display */ }
    {error && (
      <div style={styles.errorBox}>
        {error}
      </div>
    )}

    { /* Result Display */ }
    {result && mode === 'embed' && (
      <div style={styles.resultBox}>
        <h3>✅ Data Hidden Successfully!</h3>
        {result.image && (
          <img src={result.image} alt="Stego" style=
{styles.resultImage} />
        )}
        <a href={`http://localhost:5000${result.downloadUrl}`}
download>

```

```

        <button style={styles.downloadButton}>
            <Download size={18} /> Download Stego Image
        </button>
    </a>
</div>
)}

{result && mode === 'extract' && (
    <div style={styles.resultBox}>
        <h3>✅ Secret Extracted!</h3>
        <div style={styles.secretDisplay}>
            <strong>Hidden Message:</strong>
            <p>{result.secret}</p>
        </div>
    </div>
)}

{/* Info Footer */}
<div style={styles.footer}>
    <Zap size={16} color="#6366f1" />
    <span>Zero Error Guarantee | AES-256 + Reed-Solomon +
LSB</span>
</div>
</div>
</div>
);
}

const styles = {
    container: {
        minHeight: '100vh',
        backgroundColor: '#0f172a',
        display: 'flex',
        justifyContent: 'center',
        alignItems: 'center',
        padding: '20px',
    },
    card: {
        backgroundColor: '#1e293b',
        borderRadius: '16px',
        padding: '40px',
        maxWidth: '600px',
        width: '100%',
        boxShadow: '0 25px 50px rgba(0,0,0,0.5)',
    },
    header: {
        textAlign: 'center',
        marginBottom: '30px',
    },
    title: {
        color: '#f1f5f9',

```

```
    fontSize: '28px',
    margin: '10px 0',
  },
  subtitle: {
    color: '#94a3b8',
    fontSize: '14px',
  },
  toggleContainer: {
    display: 'flex',
    gap: '10px',
    marginBottom: '20px',
  },
  toggleButton: {
    flex: 1,
    padding: '12px',
    border: 'none',
    borderRadius: '8px',
    color: 'white',
    cursor: 'pointer',
    fontSize: '16px',
    display: 'flex',
    alignItems: 'center',
    justifyContent: 'center',
    gap: '8px',
  },
  uploadArea: {
    marginBottom: '20px',
  },
  uploadLabel: {
    cursor: 'pointer',
    display: 'block',
  },
  preview: {
    width: '100%',
    maxHeight: '300px',
    objectFit: 'contain',
    borderRadius: '8px',
  },
  uploadPlaceholder: {
    border: '2px dashed #475569',
    borderRadius: '8px',
    padding: '40px',
    textAlign: 'center',
    color: '#64748b',
  },
  textarea: {
    width: '100%',
    padding: '12px',
    borderRadius: '8px',
    border: '1px solid #475569',
    backgroundColor: '#0f172a',
```

```
    color: '#f1f5f9',
    fontSize: '14px',
    marginBottom: '20px',
    resize: 'vertical',
  },
  input: {
    width: '100%',
    padding: '12px',
    borderRadius: '8px',
    border: '1px solid #475569',
    backgroundColor: '#0f172a',
    color: '#f1f5f9',
    fontSize: '14px',
    marginBottom: '20px',
  },
  submitButton: {
    width: '100%',
    padding: '14px',
    backgroundColor: '#6366f1',
    color: 'white',
    border: 'none',
    borderRadius: '8px',
    fontSize: '16px',
    fontWeight: 'bold',
    cursor: 'pointer',
    display: 'flex',
    alignItems: 'center',
    justifyContent: 'center',
    gap: '8px',
    marginBottom: '20px',
  },
  errorBox: {
    backgroundColor: '#7f1d1d',
    color: '#fca5a5',
    padding: '12px',
    borderRadius: '8px',
    marginBottom: '20px',
  },
  resultBox: {
    backgroundColor: '#14532d',
    padding: '20px',
    borderRadius: '8px',
    marginBottom: '20px',
    color: '#bbf7d0',
  },
  resultImage: {
    width: '100%',
    borderRadius: '8px',
    margin: '10px 0',
  },
  downloadButton: {
```

```

        backgroundColor: '#6366f1',
        color: 'white',
        border: 'none',
        padding: '10px 20px',
        borderRadius: '6px',
        cursor: 'pointer',
        display: 'flex',
        alignItems: 'center',
        gap: '8px',
        marginTop: '10px',
    },
    secretDisplay: {
        backgroundColor: '#0f172a',
        padding: '12px',
        borderRadius: '6px',
        marginTop: '10px',
        color: '#f1f5f9',
    },
    footer: {
        textAlign: 'center',
        color: '#64748b',
        fontSize: '12px',
        display: 'flex',
        alignItems: 'center',
        justifyContent: 'center',
        gap: '8px',
        marginTop: '20px',
    },
};

export default App;

```

server.js

```

const express = require('express');
const multer = require('multer');
const cors = require('cors');
const { exec } = require('child_process');
const path = require('path');
const fs = require('fs');
const { v4: uuidv4 } = require('uuid');

const app = express();
app.use(cors());
app.use(express.json());

// File upload setup
const storage = multer.diskStorage({
    destination: './uploads',

```



```

    filename: (req, file, cb) => {
      cb(null, uuidv4() + path.extname(file.originalname));
    }
  });
const upload = multer({ storage });

// Ensure directories exist
['uploads', 'outputs'].forEach(dir => {
  if (!fs.existsSync(dir)) fs.mkdirSync(dir);
});

// Embed endpoint
app.post('/api/embed', upload.single('image'), (req, res) => {
  try {
    const { secret, password } = req.body;
    const imagePath = req.file.path;
    const outputPath = `outputs/${uuidv4()}.png`;

    const command = `python ../python-engine/steganography.py embed
"${imagePath}" "${secret}" "${password}" "${outputPath}"`;

    exec(command, (error, stdout, stderr) => {
      if (error) {
        return res.status(500).json({ error: stderr });
      }

      const result = JSON.parse(stdout);

      // Read output image and send as base64
      const imageBuffer = fs.readFileSync(outputPath);
      const base64Image = imageBuffer.toString('base64');

      res.json({
        success: true,
        image: `data:image/png;base64,${base64Image}`,
        downloadUrl: `/api/download/${path.basename(outputPath)}`
      });

      // Cleanup
      fs.unlinkSync(imagePath);
    });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// Extract endpoint
app.post('/api/extract', upload.single('stegoImage'), (req, res) => {
  try {
    const { password } = req.body;
    const stegoPath = req.file.path;

```

```

    const command = `python ../python-engine/steganography.py extract
"${stegoPath}" "${password}"`;

    exec(command, (error, stdout, stderr) => {
        fs.unlinkSync(stegoPath); // Cleanup

        if (error) {
            return res.status(500).json({ error: 'Extraction failed. Wrong
password or corrupted image.' });
        }

        const result = JSON.parse(stdout);
        res.json({ success: true, secret: result.secret });
    });
} catch (error) {
    res.status(500).json({ error: error.message });
}
});

// Download endpoint
app.get('/api/download/:filename', (req, res) => {
    const filePath = path.join(__dirname, 'outputs',
req.params.filename);
    res.download(filePath);
});

app.listen(5000, () => {
    console.log('Backend running on port 5000');
});

```